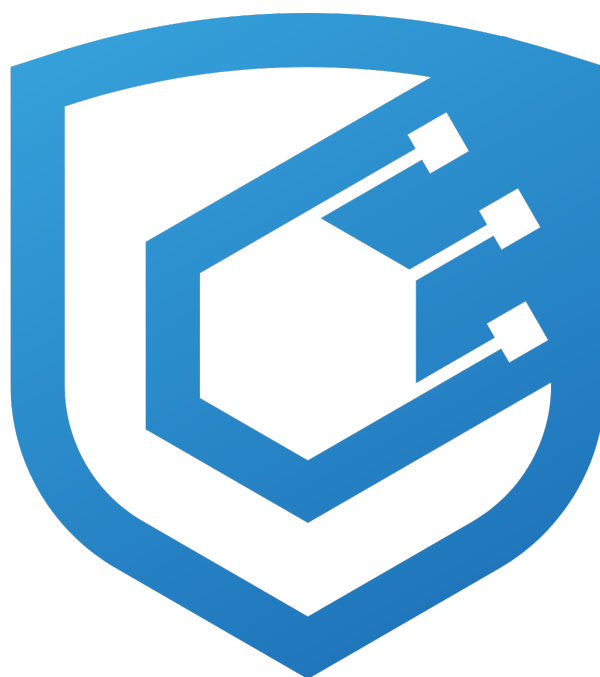




Securitize SVM VaultRegistrar SPL ACL Audit Report

Prepared by [Cyfrin](#)
Version 2.0

Lead Auditors

[JesJupyter](#)
[Nate](#)
[Ctrus](#)

September 17, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Identifier Provenance	3
6.2	Previously Audited DS Path	3
6.3	Pinned SPL ACL ALLOWED_INITIALIZERS Confirmation	4
6.4	Centralization Risks	4
6.5	Post-Engagement Changes	4
7	Findings	6
7.1	Low Risk	6
7.1.1	Unprivileged pre-funding of the registrar authority PDA can cause a retry-stable denial of SPL vault registration	6
7.1.2	Revoking an SPL mint authority permanently prevents subsequent registrar initialization	7
7.1.3	Frozen investors can still pass the participation gate and register a new vault	8
7.1.4	An investor can pre-attach a predictable DS vault and block its Registrar registration	10
7.1.5	Registrar admin can thaw vaults belonging to another registrar	13
7.2	Informational	14
7.2.1	Permissionless initialize derives the registrar state and authority PDAs from a shared monotonic counter, letting any caller front-run and invalidate pre-signed setup transactions	14
7.2.2	Operators can thaw a frozen vault ATA through register_vault_spl under extreme cases	14
7.2.3	thaw_vault_token_account_spl unconditionally CPIs ACL thaw_account and reverts on an already-thawed vault token account, unlike the guarded register_vault_spl path	15
7.2.4	SPL investor token account is not explicitly restricted to Token-2022	15
7.2.5	VaultRegisteredDs can report a signer unrelated to the attached identity	16
7.2.6	VaultRegistrarInitialized omits the initial require_investor_signature value	17
7.3	Gas Optimization	19
7.3.1	Cache repeated access_control_authority_pda and mint_config_pda derivations to cut redundant hashing in the SPL register and thaw flow	19

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Vault Registrar is a Solana program that links a protocol vault to an investor's on-chain compliance record, allowing the vault to hold regulated tokens. The program does not custody those assets; they remain in the vault's token account.

The registrar originally supported DS tokens through `rwa-rbac` and the identity registry. This engagement adds a separate path for SPL Token-2022 mints managed by the SPL ACL, with support for both direct ACL and sRFC-37 freeze authorities. SPL registration copies the identifier from the investor's whitelist record to a new record for the vault, creates the vault's associated token account if needed, and thaws it if frozen. At initialization, each registrar is classified as either DS or SPL, and its instructions are restricted to the matching path.

Only addresses in the registrar `ALLOWED_INITIALIZERS` list may create a registrar and become its admin. The admin can later be handed over in a single step with `change_admin`. The registrar admin and approved operators can register vaults. Depending on the registrar's settings, the investor may also be required to sign and hold a non-zero token balance. Operators may thaw a registered vault token account only when it has a zero balance; thawing an account that still holds tokens is admin-only.

5 Audit Scope

The audit scope was limited to:

```
// new files
programs/vault-registrar/src/instructions/cpi/acl/mod.rs
programs/vault-registrar/src/instructions/cpi/acl/thaw_account.rs
programs/vault-registrar/src/instructions/cpi/mod.rs
programs/vault-registrar/src/instructions/cpi/whitelist/create_investor_registry.rs
programs/vault-registrar/src/instructions/cpi/whitelist/mod.rs
programs/vault-registrar/src/instructions/register_vault_spl.rs
programs/vault-registrar/src/instructions/shared/acl_thaw_accounts.rs
programs/vault-registrar/src/instructions/shared/mod.rs
```

```

programs/vault-registrar/src/instructions/shared/whitelist_investor_registry_accounts.rs
programs/vault-registrar/src/instructions/thaw_vault_token_account_spl.rs
programs/vault-registrar/src/states/token_config.rs
programs/vault-registrar/src/utils/access_control.rs
programs/vault-registrar/src/utils/freeze_authority_type.rs
programs/vault-registrar/src/utils/investor_registry.rs
programs/vault-registrar/src/utils/mint.rs
programs/vault-registrar/src/utils/mod.rs
programs/vault-registrar/src/utils/pda.rs

// modified existing files
programs/vault-registrar/src/constants.rs
programs/vault-registrar/src/errors.rs
programs/vault-registrar/src/events.rs
programs/vault-registrar/src/instructions/admin/add_operator.rs
programs/vault-registrar/src/instructions/admin/initialize.rs
programs/vault-registrar/src/instructions/admin/is_operator.rs
programs/vault-registrar/src/instructions/mod.rs
programs/vault-registrar/src/instructions/register_vault_ds.rs
programs/vault-registrar/src/lib.rs
programs/vault-registrar/src/states/mod.rs
programs/vault-registrar/src/states/vault_registrar_state.rs

```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [SVM VaultRegistrar SPL ACL](#) code provided by [Securitize](#). The findings consist of 5 Low severity issues with the remainder being informational and gas optimizations.

This engagement reviewed the changes to the previously audited DS-only Vault Registrar. The scope included the new SPL Token-2022 registration flow, the standalone thaw instruction, DS/SPL configuration at initialization, and the related changes to the existing DS and admin instructions.

We reviewed and compared the DS and SPL registration workflows and their requirements side by side. For each finding, we also checked whether the same issue applied to the other path.

6.1 Identifier Provenance

We found no SPL equivalent of the identifier-substitution issue reported as Medium in the March *Securitize Solana Whitelister* audit. `register_vault_spl` derives the investor's canonical `InvestorRegistry` from the mint and investor wallet, verifies the account owner and its recorded mint and wallet, and then copies the stored `investor_id` to the vault's registry. The identifier is never supplied by the caller, and a registry belonging to another wallet cannot be substituted.

When `require_investor_signature` is enabled, the same investor wallet must sign and present a non-zero token balance. The finding concerning frozen investors weakens this participation check, but it does not affect the source or binding of the identifier. When the policy is disabled, the authorized caller is trusted to select the intended investor, as described in the protocol's trust assumptions.

6.2 Previously Audited DS Path

The changes to `register_vault_ds`, `add_operator`, and `is_operator` are largely structural: interface renaming, extraction of a shared authorization helper, addition of the DS type guard, and minor error-handling and formatting changes. We found no authorization or identity-binding regression introduced by these edits. With respect to the behavior changed in this engagement, they preserve the conclusions of the March audit.

The side-by-side review did identify two DS issues: the Low first-writer denial of service against predictable vault addresses and an Informational attribution issue in the `VaultRegisteredDs` event.

We also reviewed the compatibility problem introduced by the extension of `VaultRegistrarState`. A legacy registrar using the full operator capacity of the old layout could fail to deserialize after the upgrade. The team confirmed that no affected account exists on mainnet (only testnet accounts are affected), so this was not reported as a separate finding.

6.3 Pinned SPL ACL `ALLOWED_INITIALIZERS` Confirmation

The only program-source change between the audited SPL ACL fix revision (47cd246636a5) and the revision pinned by the registrar (4ef774e27a836f56e2048becfd4f6d6f54ceb0b3) rotates the four keys in the ACL `ALLOWED_INITIALIZERS` list. The allowlist mechanism, its length, and the `devnet` behavior are unchanged. That constant is used only when an ACL instance is created; it does not affect freeze-role checks or any CPI used by the registrar. This change does not alter the conclusions of the previous audit.

6.4 Centralization Risks

In production, `initialize` is gated by the registrar `ALLOWED_INITIALIZERS` list: only an allowlisted key may sign as `admin` and create a registrar. A registrar remains inactive until the token administrator grants its PDA the required ACL or RBAC role. The registrar admin manages operators, pausing, the investor-signature policy, and standalone SPL thawing, and the admin key can be rotated through one-step transfer and can be unrecoverable if the admin key is lost or transferred to an incorrect address. When investor signatures are optional, operators are trusted to associate vaults with the intended investors. The system therefore relies on the token administrator, registrar admin, authorized operators, and program upgrade authority to use their privileges correctly.

6.5 Post-Engagement Changes

On September 16th, 2026, the team submitted a further change that restricted `initialize` to the registrar `ALLOWED_INITIALIZERS` list. `initialize` was originally permissionless, as documented in 99db3ea. The same change added a `change_admin` instruction (located in `instructions/admin/change_admin.rs`) to update the admin address. Cyfrin reviewed and confirmed both changes. `change_admin` was introduced after the original engagement and is not part of the original audit scope.

Summary

Project Name	SVM VaultRegistrar SPL ACL
Repository	bc-solana-whitelister
Commit	629a80e7966d...
Fix Commit	44aef5dd6e0f...
Audit Timeline	Aug 25th - Aug 28th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	5
Informational	6
Gas Optimizations	1
Total Issues	12

Summary of Findings

[L-1] Unprivileged pre-funding of the registrar authority PDA can cause a retry-stable denial of SPL vault registration	Resolved
[L-2] Revoking an SPL mint authority permanently prevents subsequent registrar initialization	Resolved
[L-3] Frozen investors can still pass the participation gate and register a new vault	Resolved
[L-4] An investor can pre-attach a predictable DS vault and block its Registrar registration	Acknowledged
[L-5] Registrar admin can thaw vaults belonging to another registrar	Resolved
[I-1] Permissionless <code>initialize</code> derives the registrar state and authority PDAs from a shared monotonic counter, letting any caller front-run and invalidate pre-signed setup transactions	Resolved
[I-2] Operators can thaw a frozen vault ATA through <code>register_vault_spl</code> under extreme cases	Resolved
[I-3] <code>thaw_vault_token_account_spl</code> unconditionally CPIs <code>ACL thaw_account</code> and reverts on an already-thawed vault token account, unlike the guarded <code>register_vault_spl</code> path	Resolved
[I-4] SPL investor token account is not explicitly restricted to Token-2022	Resolved
[I-5] <code>VaultRegisteredDs</code> can report a signer unrelated to the attached identity	Resolved
[I-6] <code>VaultRegistrarInitialized</code> omits the initial <code>require_investor_signature</code> value	Resolved
[G-1] Cache repeated <code>access_control_authority_pda</code> and <code>mint_config_pda</code> derivations to cut redundant hashing in the SPL register and thaw flow	Resolved

7 Findings

7.1 Low Risk

7.1.1 Unprivileged pre-funding of the registrar authority PDA can cause a retry-stable denial of SPL vault registration

Description: Note: SPL only. `register_vault_ds` does not call `fund_authority_for_rent` or use the registrar authority PDA as a rent payer; it forwards the transaction payer directly to the DS CPI. Therefore, the pre-funded PDA residue mechanism described below does not affect DS registration.

Affected path: SPL only. `register_vault_ds` does not call `fund_authority_for_rent` or use the registrar authority PDA as a rent payer; it forwards the transaction payer directly to the DS CPI. Therefore, the pre-funded PDA residue mechanism described below does not affect DS registration.

`register_vault_spl` uses the System-owned `vault_registrar_authority` PDA as both signer and rent payer for the whitelist CPI that creates a vault's `InvestorRegistry`. Immediately before the CPI, `fund_authority_for_rent` tops the PDA up to the expected account-creation rent:

```
fn fund_authority_for_rent(ctx: &Context<RegisterVaultSpl>) -> Result<> {
    let authority = ctx.accounts.vault_registrar_authority.to_account_info();

    let required = Rent::get()?.minimum_balance(INVESTOR_REGISTRY_ACCOUNT_LEN);
    let shortfall = required.saturating_sub(authority.lamports());

    if shortfall == 0 {
        return Ok(());
    }

    let cpi_accounts = Transfer {
        from: ctx.accounts.payer.to_account_info(),
        to: authority,
    };

    transfer(
        CpiContext::new(ctx.accounts.system_program.to_account_info(), cpi_accounts),
        shortfall,
    )
}
```

The calculation assumes the PDA begins at or below `required`, but its address is publicly derivable and anyone can transfer lamports to it. An attacker can pre-fund it with `required + 1`; `saturating_sub` then returns zero, so the registrar performs no top-up. If `spl_token_whitelist::create_investor_registry` debits the full fresh-account rent, the PDA is left with one lamport:

```
pre-CPI authority balance = required + 1
registrar top-up         = 0
whitelist account rent   = required
post-CPI authority balance = 1
```

Because the authority is writable, System-owned, and has zero data, it begins the registration rent-exempt but would finish rent-paying. Agave rejects this `RentExempt -> RentPaying` transition with `InsufficientFundsForRent`. The transaction rolls back and restores the `required + 1` balance, so ordinary retries fail identically until another transfer moves the PDA outside the rent-paying residue band.

The registrar arithmetic and Agave v3.1.3 rent rule are verified. The remaining dependency gate is the locked private whitelist: final confirmation requires establishing that deployed `create_investor_registry` uses the authority as payer and debits the full fresh-account rent. The local interface and documentation say `signer` is the payer, but do not verify the unavailable implementation's internal behavior.

Impact: Any funded account can temporarily block all new `register_vault_spl` operations for a targeted registrar. While the PDA remains in the poisoned range, new vaults cannot obtain their compliance registry; on default-frozen

mints, they also cannot complete the create-and-thaw onboarding flow.

Recovery is permissionless. For the required + 1 trigger, adding `Rent::minimum_balance(0) - 1` lamports makes the post-CPI balance exactly `Rent::minimum_balance(0)`, allowing the authority to remain rent-exempt and registration to complete.

The registrar is therefore not permanently bricked, and a correctly diagnosed top-up restores service cheaply. However, ordinary retries do not self-heal, the runtime error does not identify the poisoned PDA, and an attacker can repeatedly re-arm public registrar authorities.

Recommended Mitigation: Ensure the authority's post-CPI balance is always either zero or rent-exempt. Possible approaches include:

- After the whitelist CPI, use the registrar authority seeds to transfer any residual lamports back to the transaction payer, leaving the authority at zero before transaction-level rent validation.
- Before the CPI, normalize an externally funded balance and then fund exactly the amount the CPI will debit.
- If a residual must remain, top the authority up so its expected post-CPI balance is at least `Rent::minimum_balance(0)` rather than inside the rent-paying interval.
- Preferably, update the whitelist interface to separate authorization from payment so the transaction payer funds `InvestorRegistry` creation directly and the authority PDA never acts as a rent transit account.

Securitize: Fixed in commit [17eedbb](#).

Cyfrin: Verified.

7.1.2 Revoking an SPL mint authority permanently prevents subsequent registrar initialization

Description: The updated `Initialize` accounts require `asset_mint` to have a live mint authority matching the supplied `asset_mint_authority` account:

```
#[account(mint::authority = asset_mint_authority)]
pub asset_mint: InterfaceAccount<'info', anchor_spl::token_interface::Mint>,

/// CHECK: Bound to the mint by the constraint above. Its owner and contents decide whether the
/// registrar is created for a DS or an SPL token.
pub asset_mint_authority: UncheckedAccount<'info>,
```

Anchor 0.31.1 expands `mint::authority` into a comparison against `COption::Some(asset_mint_authority.key())`:

```
if asset_mint.mint_authority
    != COption::Some(asset_mint_authority.key())
{
    return Err(ErrorCode::ConstraintMintMintAuthority.into());
}
```

No account can satisfy this constraint when the Token-2022 mint's authority has been permanently revoked to `COption::None`. Account validation therefore fails before `initialize_handler` reaches `detect_token_config` and classifies the mint as SPL.

This live mint-authority requirement is unnecessary for the SPL branch itself. After classification, `require_spl_mint_accounts` validates that the account is a Token-2022 mint and that its freeze authority is controlled by the ACL:

```
fn require_spl_mint_accounts(
    ctx: &Context<Initialize>,
    asset_mint: &Pubkey,
) -> Result<()> {
    require!(
        ctx.accounts.identity_registry.is_none(),
        VaultRegistrarError::IdentityRegistryNotAllowed
    )
}
```



```

);

require_token_2022_mint(&ctx.accounts.asset_mint)?;

let freeze_authority_type =
    FreezeAuthorityType::from_mint(&ctx.accounts.asset_mint)?;
let mint_config = ctx.accounts.mint_config.as_ref().map(|a| a.as_ref());

freeze_authority_type.require_mint_config(asset_mint, mint_config)
}

```

Later SPL registration and thaw operations likewise depend on the ACL-controlled freeze authority rather than the mint authority. The SDK independently enforces the same restriction by rejecting initialization whenever `mint.mintAuthority` is absent.

Consequently, an otherwise valid ACL-managed asset that finalizes issuance by revoking its mint authority cannot create a registrar afterward, even though its freeze authority and every account required for SPL registration and thawing remain valid. Because setting a Token-2022 authority to `None` is irreversible, the asset cannot restore the authority merely to satisfy initialization.

Impact: ACL-managed assets that revoke mint authority before registrar setup are permanently excluded from the registrar's SPL integration path. They cannot onboard vaults through a newly created registrar unless the program is upgraded or the registrar was initialized before revocation.

The issue requires no attacker, does not affect registrars created before revocation, and depends on whether authority revocation is a supported asset lifecycle. If every current and planned asset intentionally retains a live mint authority, this is instead a deployment prerequisite that should be documented and enforced operationally.

Recommended Mitigation: Decouple SPL classification from the existence of a mint authority:

- Make `asset_mint_authority` optional and remove the unconditional `mint::authority` account constraint.
- When `asset_mint.mint_authority` is `Some`, require the supplied authority account to match before using it for DS classification.
- When it is `None`, permit only the SPL branch and continue enforcing Token-2022 ownership and the ACL-backed freeze-authority checks.
- Preserve the existing DS invariant that a DS mint must have the expected asset-controller mint authority.
- Update the SDK so a missing mint authority is passed as the optional account state instead of rejected client-side.

Securitize: Fixed in commit [a4d0208](#).

Cyfrin: Verified.

7.1.3 Frozen investors can still pass the participation gate and register a new vault

Description: When `require_investor_signature` is on, both `register_vault_spl` and `register_vault_ds` require the investor to sign and to present a token account with `amount > 0`. Neither path reads `state`.

SPL (`require_investor_participation`):

```

fn require_investor_participation(ctx: &Context<RegisterVaultSpl>) -> Result<()> {
    if !ctx
        .accounts
        .vault_registrar_state
        .require_investor_signature
    {
        return Ok(());
    }

    require!(
        ctx.accounts.investor_wallet.to_account_info().is_signer,

```

```

        VaultRegistrarError::InvestorAccountsRequired
    );

    let investor_token_account = ctx
        .accounts
        .investor_token_account
        .as_ref()
        .ok_or(VaultRegistrarError::InvestorAccountsRequired)?;

    require!(
        investor_token_account.amount > 0,
        VaultRegistrarError::InvestorHasNoBalance
    );

    Ok(())
}

```

DS (register_vault_ds):

```

if vault_registrar_state.require_investor_signature {
    require!(
        ctx.accounts.existing_investor_wallet.is_some()
            && ctx.accounts.existing_investor_wallet_identity.is_some()
            && ctx.accounts.investor_token_account.is_some(),
        VaultRegistrarError::InvestorAccountsRequired
    );

    let investor_token_account = ctx.accounts.investor_token_account.as_ref().unwrap();

    require!(
        investor_token_account.amount > 0,
        VaultRegistrarError::InvestorHasNoBalance
    );
}

```

Freezing a Token/Token-2022 account does not clear its balance, so a legitimately frozen investor ATA still passes. After that:

- SPL copies investor_id into a new vault registry and thaws the new vault ATA when the mint uses DefaultAccountState (new accounts start frozen):

```

if ctx.accounts.vault_token_account.state != AccountState::Frozen {
    return Ok(());
}

cpi::acl::thaw_account::handler(
    &ctx.accounts.acl_thaw_accounts,
    ctx.accounts.vault_registrar_authority.to_account_info(),
    ctx.accounts.asset_mint.to_account_info(),
    ctx.accounts.vault_token_account.to_account_info(),
    ctx.accounts.token_program.to_account_info(),
    state.id,
    state.authority_bump,
)?;

```

Impact: The freeze stays on the original account. Those tokens still cannot move. What the investor gets is a new path: the SPL vault ATA is usable, or the DS wallet is newly attached, and future tokens can land there.

Recommended Mitigation: Reject `investor_token_account.state == Frozen` (require `Initialized`) in both registration paths.

Securitize: Fixed in commit [dc71c35](#).

Cyfrin: Verified.

7.1.4 An investor can pre-attach a predictable DS vault and block its Registrar registration

Description: Note: DS only. This is found during the path comparison between DS and SPL token.

The bug is who creates `WalletIdentity` first. Identity Registry owns a unique PDA `WalletIdentity(wallet, mint)` and inits it. Vault Registrar, `rwa-rbac`, and `rwa-imr` all write that same account. The registrar assumes an existing record is a prior registration. An investor can create it earlier through IMR, without going through the registrar at all. SPL registration uses whitelist `InvestorRegistry` and does not have this path.

A registrar instance is created with `initialize`, for one mint. It stores `admin`, `asset_mint`, and `operators` (max 10). The instance cannot register DS vaults until the DS token admin assigns RWA RBAC user role id to the **registrar state PDA**. That grant is outside `initialize`.

[programs/vault-registrar/src/instructions/admin/initialize.rs](#) · [docs/VAULT_REGISTRAR_GUIDE.md §6.1](#)

```
vault_registrar program
  VaultRegistrarState (PDA ["vault_registrar_state", id], one mint)
    admin
    operators          ← DeFi protocols that may call register_vault_ds
    asset_mint
```

Two different “roles”:

- Registrar **operator**: a key that is allowed to call `register_vault_ds`.
- RBAC **user role id 1** on the **state PDA**: what lets that PDA pass `ATTACH_WALLET_TO_IDENTITY` when it CPIs into `rwa-rbac`.

1. Normal registration

Protocol (operator) → Vault Registrar → `rwa-rbac` → Identity Registry.

Demo passes its singleton vault PDA into the registrar:

[programs/demo_defi_protocol/src/instructions/register_vault_ds.rs](#)

That vault address is `PDA(["vault"], BhPDdnSU41tVqCChVSkLqdzJgmh3vr9ER7U1b39oWwzX)`. Program id and seed are public, so the address is known before `demo_defi_protocol::initialize` creates the account:

[programs/demo_defi_protocol/src/lib.rs](#) · [initialize.rs](#)

```
declare_id!("BhPDdnSU41tVqCChVSkLqdzJgmh3vr9ER7U1b39oWwzX");

#[account(
  init,
  payer = payer,
  space = 8 + VaultState::INIT_SPACE,
  seeds = [b"vault"],
  bump
)]
pub vault: Account<'info, VaultState>,
```

Registrar checks `pause` + authorized caller, derives the same `WalletIdentity` PDA, and attaches only if it is still empty:

[programs/vault-registrar/src/instructions/register_vault_ds.rs](#)

```
constraint = !vault_registrar_state.paused,
constraint = vault_registrar_state.is_authorized_caller(&caller.key()),

let (expected_vault_wallet_identity, _) = Pubkey::find_program_address(
  &
```

```

        ctx.accounts.vault_wallet.key().as_ref(),
        ctx.accounts.asset_mint.key().as_ref(),
    ],
    ctx.accounts.identity_registry_program.key,
);

if !vault_wallet_identity_info.data_is_empty() {
    // same identity → VaultAlreadyRegistered
    // other identity → VaultBelongsToDifferentInvestor
}

rwa_rbac::cpi::attach_wallet_to_identity(
    /* user = vault_registrar_state, signed with registrar seeds */,
    cpi_data,
)?;

```

First hop: the **state PDA** signs as user. rwa-rbac checks ATTACH_WALLET_TO_IDENTITY on that PDA. This is why the role must already have been granted.

Second hop: rwa-rbac signs as controller_authority with controller_seeds ([controller.key, bump]) and CPIs into Identity Registry. That PDA is identity_registry.authority.

[programs/rwa-rbac/.../attach_wallet_to_identity.rs](#)

```

Protocol operator
→ register_vault_ds
    → rwa-rbac attach (user = VaultRegistrarState, role already granted)
        → Identity Registry attach (authority = controller_authority)
            → init WalletIdentity

```

Documented CPI depth: Protocol → VaultRegistrar → rwa-rbac → Identity Registry. [docs/INTEGRATION_GUIDE_DS.md](#)

Identity Registry then inits WalletIdentity(wallet, mint):

```

#[account(
    init,
    seeds = [wallet.key().as_ref(), asset_mint.key().as_ref()],
    payer = payer,
    space = 8 + WalletIdentity::INIT_SPACE,
    bump,
)]
pub wallet_identity: Box<Account<'info, WalletIdentity>>;

```

That init is first-writer-wins. The authority Identity Registry actually checks is not only the controller. It accepts either identity_registry.authority (the controller path above) **or** identity_account.owner (the investor PDA):

```

require!(
    ctx.accounts.authority.key() == ctx.accounts.identity_account.owner
    || ctx.accounts.authority.key() == ctx.accounts.identity_registry.authority,
    IdentityRegistryErrors::UnauthorizedSigner
);

```

[programs/identity_registry/.../attach_wallet_to_identity.rs](#)

2. What a normal investor can do

They never touch the registrar. A registered investor calls IMR attach_wallet_by_investor and passes the predictable vault address as new_wallet. Public entrypoint: [programs/rwa-imr/src/lib.rs](#)

The caller must already be an investor on that mint and must sign with a wallet already attached to their identity. Random keys cannot do this:

[programs/rwa-imr/.../attach_wallet_by_investor.rs](#)

```
pub wallet: Signer<'info>,
/// CHECK: For wallet to be added. Verified in CPI
pub new_wallet_identity: UncheckedAccount<'info>,

pub fn handler(ctx: Context<AttachWalletByInvestor>, new_wallet: Pubkey) -> Result<()> {
    let wallet_identity = WalletIdentity::deserialize_checked(&ctx.accounts.wallet_identity)?;
    require_keys_eq!(wallet_identity.identity_account, ctx.accounts.identity_account.key());
    require_keys_eq!(ctx.accounts.wallet.key(), wallet_identity.wallet);
```

`new_wallet` is only an instruction argument. The target does not sign, does not need to exist, and may be another program's PDA:

```
let mut cpi_data = ATTACH_WALLET_TO_IDENTITY_IX.to_vec();
cpi_data.extend(new_wallet.as_ref());

AccountMeta::new_readonly(ctx.accounts.investor.key(), true), // investor PDA = identity_account.owner
invoke_signed(/* Identity Registry attach */, signers_seeds)?;
```

Same file L42–L99

This transaction never enters Vault Registrar, so `pause`, `operator`, and `require_investor_signature` are all skipped. Identity Registry accepts IMR's investor-PDA signature as `identity_account.owner` — the left-hand branch. No RBAC role, no `controller_seeds`.

If `WalletIdentity` is already initialized, IR `init` fails. The race is to land before the legitimate `register_vault_ds`.

When the operator later registers, the registrar sees a non-empty account: wrong identity → `VaultBelongsToDifferentInvestor`; same identity → `VaultAlreadyRegistered`. It does not CPI attach and does not emit `VaultRegisteredDs`.

Registrar L122–L132 · tests for those two errors (after a successful registrar write, same branch): [spec L934–L990](#)

```
Investor (already registered, signs an attached wallet)
→ IMR attach_wallet_by_investor(new_wallet = vault PDA)
  → Identity Registry attach (authority = investor PDA)
    → init WalletIdentity ← occupies the account the registrar needed
```

Note: For SPL path, a freeze authority or whitelist admin *can* pre-create the registry. But that is a privileged write they already have.

Impact: If the vault address is predictable — as in the demo, `PDA(["vault"])` — any other registered investor can occupy it. Investor B attaches that address to B's own identity through IMR before the protocol registers it for investor A. `register_vault_ds` then returns `VaultBelongsToDifferentInvestor`, so A cannot onboard the protocol.

That is the poison: the `WalletIdentity` slot for the vault is taken. B does not get the vault's key or its tokens; A just cannot complete registration on that address.

Note: For SPL path, a freeze authority or whitelist admin *can* pre-create the registry. But that is a privileged write they already have.

Recommended Mitigation: Either:

1. On `attach_wallet_by_investor`, require proof of control of `new_wallet` (the wallet signs, or the owning program CPI-signs for a PDA). Do not accept an unauthenticated pubkey.
2. Document that a predictable vault address can be bound by another investor before registrar onboarding. Treat those cases as an operational/compliance issue: watch for them and Perform some actions on the malicious address.

Securitize: Acknowledged and documented in commit [c16281c](#).

7.1.5 Registrar admin can thaw vaults belonging to another registrar

Description: Multiple registrars may exist for the same mint, each with a different authority PDA as documented:

```
## 5. Create a registrar
```

One registrar serves one mint. Several registrars may exist **for** the same mint; each gets its own id and its own PDAs.

However, `thaw_vault_token_account_spl` only verifies that:

- the caller is the admin of the supplied registrar;
- the mint matches that registrar;
- the vault has a valid global InvestorRegistry. and few more checks but registrar does not prove that the vault belongs to that specific registrar. Registrar A's admin can pass B's vaults to `thaw_vault_token_account_spl` and A can remove the frozen state from B's vault, potentially allowing it to receive or transfer tokens despite B's intended compliance or administrative freeze.

Impact: Registrar A's admin can:

1. Use Registrar A's state and authority PDA.
2. Pass Registrar B's vault wallet and token account.
3. Pass the canonical registry for (mint, B's vault).
4. Cause Registrar A's authority to thaw B's frozen vault. This can reverse a freeze imposed on B's vault and allow it to receive or transfer tokens. It also causes the emitted thaw event to attribute the action to Registrar A, even though the vault may have been registered through Registrar B. This broadens the admin's effective freeze authority beyond the documented vault-repair use case.

Recommended Mitigation: Bind every vault to the registrar that registered it.

Securitize: Fixed in commit [de22499](#).

Cyfrin: Verified.

7.2 Informational

7.2.1 Permissionless `initialize` derives the registrar state and authority PDAs from a shared monotonic counter, letting any caller front-run and invalidate pre-signed setup transactions

Description: `initialize` is permissionless: the `Initialize` accounts struct carries no access-control constraint, and `initialize_handler` records `admin` without ever checking the caller. The per-instance `vault_registrar_state` PDA and the `vault_registrar_authority` PDA are both seeded from a single global `vault_registrar_counter` count: the state at seeds = `[VAULT_REGISTRAR_STATE_SEED, vault_registrar_counter.count.to_le_bytes()]` and the authority analogously, with the counter advanced on every call. Because any caller can advance the counter, the address every subsequently pre-derived registrar PDA maps to changes.

Impact: A token issuer that reads the counter, derives its registrar PDA, and pre-signs an `initialize` transaction has that transaction rejected the moment anyone else's `initialize` lands first and advances the counter. The issuer loses the submitted transaction fee and must re-read and re-sign; the registrar address is recoverable by re-derivation, but a persistent adversary can deny first-try success repeatedly for only the cost of one extra registrar's rent.

Recommended Mitigation: Derive the registrar state and authority PDAs from immutable per-issuer data that an unrelated party cannot front-run. Seeding only with `asset_mint` leaves `initialize` permissionless and lets any caller record themselves as `admin` of the single per-mint registrar, so either gate `initialize` behind an authorized mint-side signer or seed the PDAs with a signer-bound component such as the intended `admin` plus the mint. Keep the global counter only if clients are documented to derive-and-retry and its advancement is gated so an unrelated party cannot invalidate a pending setup.

Securitize: Fixed in [b7ebcf17](#) and documented in [99db3ea](#).

Cyfrin: Verified.

7.2.2 Operators can thaw a frozen vault ATA through `register_vault_spl` under extreme cases

Description: `register_vault_spl` is callable by any `admin` or operator (`is_authorized_caller`), and its `thaw_vault_token_account_if_frozen` invokes the same ACL `thaw_account` CPI signed by the `vault_registrar_authority` PDA whenever the vault token account is `AccountState::Frozen`. That thaw does not distinguish a freeze imposed by the mint's `DefaultAccountState` from a freeze applied by the token administrator to a specific account. The standalone `thaw_vault_token_account_spl` gates the identical call behind `has_one = admin`, but the registration path reaches the same thaw through a lower-privileged caller for a vault that has not yet been registered.

Also, `register_vault_spl` provides an operator-accessible route to the same sensitive thaw operation that `thaw_vault_token_account_spl` deliberately reserves for the registrar `admin`. Registration accepts an existing canonical vault ATA through `init_if_needed`. After creating a missing `InvestorRegistry`, it thaws that ATA whenever it is frozen. Consequently, if compliance administrators **freeze a vault and delete its registry as part of revocation**, an operator can select the same vault, recreate its registry from another investor record, and thaw the legally or administratively frozen ATA. The registration path contains no distinction between a newly default-frozen ATA and a previously existing externally frozen ATA.

```
/// Closes the registry account and refunds rent to `receiver` (or `signer` when
/// `receiver` is omitted). Callable by admin or by a freeze authority of the mint.
pub fn delete_investor_registry_handler(ctx: Context<DeleteInvestorRegistry>) -> Result<()> {
    if ctx.accounts.signer.key() != ctx.accounts.spl_whitelist_state.admin {
        require_freeze_authority(
            &ctx.accounts.signer,
            &ctx.accounts.mint.to_account_info(),
            &ctx.accounts.freeze_authority.to_account_info(),
            ctx.accounts.srfc37_authority.as_ref().map(|a| a.as_ref()),
            ctx.accounts.access_control_state.as_ref(),
        )?;
    }

    emit!(crate::events::InvestorRegistryDeleted {
```

```

        mint: ctx.accounts.investor_registry.mint,
        wallet: ctx.accounts.investor_registry.wallet,
    });

    let rent_receiver = ctx
        .accounts
        .receiver
        .as_ref()
        .map(|r| r.to_account_info())
        .unwrap_or_else(|| ctx.accounts.signer.to_account_info());

    ctx.accounts.investor_registry.close(rent_receiver)?;

    Ok(())
}

```

Impact: The impact is limited. It's theoretically valid, but it has some strong prerequisites:

1. A vault account has been frozen and is not connected to any investor.
2. A vault account has been frozen and its registry has been revoked.

Both are strong prerequisites, so this is just as a reminder. As a result, an operator can register a not-yet-registered vault and, as a side effect, thaw its frozen associated token account, reversing a compliance or legal freeze that the registrar's own design scopes to the admin.

Recommended Mitigation: Either:

1. During the operation process, simply avoid making the vault account isolated (where it has not been connected to any investor).
2. Only permit the thaw from time-locks, with a strong validation.

Securitize: Fixed in commit: [80246a](#)

Cyfrin: Verified.

7.2.3 `thaw_vault_token_account_spl` unconditionally CPIs `thaw_account` and reverts on an already-thawed vault token account, unlike the guarded `register_vault_spl` path

Description: `thaw_vault_token_account_spl` invokes the ACL `thaw_account` CPI unconditionally without first checking whether the vault token account is frozen, whereas the `register_vault_spl` path guards the identical CPI with `thaw_vault_token_account_if_frozen` returning early unless `vault_token_account.state == AccountState::Frozen`. Because `vault_token_account` is declared `init_if_needed`, on a mint without a `DefaultAccountState` extension a missing vault token account is created already thawed, and the token-2022 `thaw_account` then rejects the already-thawed account, reverting the transaction.

Impact: The standalone thaw instruction is non-idempotent: on the default mint configuration it can never create-and-thaw a missing vault token account, breaking the administrative restore path it exists to serve. The instruction is admin-only and involves no fund loss or third-party harm, so the impact is confined to a broken admin repair flow.

Recommended Mitigation: Mirror the registration-path guard by reading `vault_token_account.state` and skipping the `thaw_account` CPI when it is not `AccountState::Frozen`.

Securitize: Fixed in commit [4d2b2b1](#).

Cyfrin: Verified.

7.2.4 SPL investor token account is not explicitly restricted to Token-2022

Description: The SPL registration instruction constrains the mint and authority of `investor_token_account`, but does not bind its owner to the instruction's fixed `Token2022` program:


```
#[account(
    token::mint = asset_mint,
    token::authority = investor_wallet,
)]
pub investor_token_account: Option<Box<InterfaceAccount<'info', TokenAccount>>>;
```

anchor_spl::token_interface::TokenAccount accepts accounts owned by either the legacy Token program or Token-2022. Without `token::token_program = token_program`, Anchor therefore does not check that this particular account is owned by Token-2022. The DS registration path includes that constraint.

Impact: The practical impact is limited to clarity: the account definition is broader than intended and differs from the equivalent DS definition.

Recommended Mitigation: Make the intended owner check explicit.

Securitize Fixed in commit [f9a347b](#).

Cyfrin: Verified.

7.2.5 VaultRegisteredDs can report a signer unrelated to the attached identity

Description: Note: This is found during the path comparison between DS and SPL token.

`existing_investor_wallet` remains an `Option<Signer>` even when `require_investor_signature` is disabled. The constraints that link this signer to the required `identity_account` are declared on the optional `existing_investor_wallet_identity` account. Anchor evaluates those constraints only when that account is provided, so omitting it skips both the `has_one = identity_account` check and the wallet-key match.

```
pub existing_investor_wallet: Option<Signer<'info>>,

#[account(
    has_one = identity_account @ VaultRegistrarError::IdentityNotOwnedByInvestor,
    constraint = existing_investor_wallet.as_ref().is_none_or(|wallet| {
        wallet.key() != ZERO_PUBKEY
    }) @ VaultRegistrarError::InvalidAddress,
    constraint = existing_investor_wallet.as_ref().is_none_or(|wallet| {
        existing_investor_wallet_identity.wallet == wallet.key()
    }) @ VaultRegistrarError::IdentityNotOwnedByInvestor,
)]
pub existing_investor_wallet_identity: Option<Account<'info', WalletIdentity>>,
```

The optional `investor_token_account` likewise constrains mint and authority only when present. Even then it binds the token account to `existing_investor_wallet`, not to `identity_account`.

The handler requires all three investor accounts only when `require_investor_signature` is enabled. With the flag disabled, an authorized operator can supply signer B as `existing_investor_wallet`, omit the identity and token accounts, and still attach the vault to an unrelated but otherwise valid `identity_account` A.

```
if vault_registrar_state.require_investor_signature {
    require!(
        ctx.accounts.existing_investor_wallet.is_some()
        && ctx.accounts.existing_investor_wallet_identity.is_some()
        && ctx.accounts.investor_token_account.is_some(),
        VaultRegistrarError::InvestorAccountsRequired
    );

    let investor_token_account = ctx.accounts.investor_token_account.as_ref().unwrap();

    require!(
        investor_token_account.amount > 0,
        VaultRegistrarError::InvestorHasNoBalance
    );
}
```

```
    );
}
```

The RWA-RBAC CPI uses the required `identity_account` and does not consume `existing_investor_wallet`. The event independently populates `investor` from `existing_investor_wallet`, so its attribution can disagree with the identity actually attached to the vault.

```
emit!(VaultRegisteredDs {
  registrar: ctx.accounts.vault_registrar_state.key(),
  caller: ctx.accounts.caller.key(),
  vault: ctx.accounts.vault_wallet.key(),
  asset_mint: ctx.accounts.asset_mint.key(),
  identity_account: ctx.accounts.identity_account.key(),
  investor: ctx
    .accounts
    .existing_investor_wallet
    .as_ref()
    .map(|w| w.key()),
});
```

1. Create a DS registrar with `require_investor_signature = false`.
2. As an authorized operator, call `register_vault_ds` with valid accounts for identity A and:
 - `existing_investor_wallet = B`, where B signs the transaction (the operator itself can be used);
 - `existing_investor_wallet_identity = None`;
 - `investor_token_account = None`.
3. The vault is attached to identity A. `VaultRegisteredDs` reports `investor = Some(B)`.

Impact: The event's `investor` field cannot reliably identify the investor associated with `identity_account` when signature enforcement is disabled. It also should not be treated as proof that the attached investor consented to the registration. Under the current trust model, it works well, but the team should be notified about this condition.

Recommended Mitigation: Emit `investor = None` when `require_investor_signature` is disabled. Alternatively, if partial investor accounts are not intended, reject them and emit the signer only after validating its `WalletIdentity` against `identity_account`.

Securitize: Fixed in commit [47c4f8](#).

Cyfrin: Verified.

7.2.6 VaultRegistrarInitialized omits the initial `require_investor_signature` value

Description: During initialization, `require_investor_signature` is correctly stored in the registrar state:

```
let state = VaultRegistrarState {
  // ...
  require_investor_signature,
  // ...
};
```

However, the emitted `VaultRegistrarInitialized` event does not include this value:

```
pub struct VaultRegistrarInitialized {
  pub registrar: Pubkey,
  pub id: u64,
  pub admin: Pubkey,
  pub asset_mint: Pubkey,
  pub token_config: TokenConfig,
  // Missing require_investor_signature
}
```

By contrast, later updates include the value in `RequireInvestorSignatureUpdated`:

```
emit!(RequireInvestorSignatureUpdated {  
  registrar: state.key(),  
  admin: ctx.accounts.admin.key(),  
  require_investor_signature,  
});
```

Recommended Mitigation: Add `require_investor_signature` to `VaultRegistrarInitialized` and include it when emitting the event:

```
pub struct VaultRegistrarInitialized {  
  pub registrar: Pubkey,  
  pub id: u64,  
  pub admin: Pubkey,  
  pub asset_mint: Pubkey,  
  pub token_config: TokenConfig,  
  pub require_investor_signature: bool,  
}
```

Securitize: Fixed in commit [c481756](#).

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 Cache repeated `access_control_authority_pda` and `mint_config_pda` derivations to cut redundant hashing in the SPL register and thaw flow

Description: `register_vault_spl` and `thaw_vault_token_account_spl` resolve the same ACL PDAs several times per execution: `FreezeAuthorityType::from_mint` derives `access_control_authority_pda` and `mint_config_pda`, and the subsequent `require_valid` validation chain re-derives those same addresses again, so a single PDA can be derived with `find_program_address` up to four times in one instruction.

```
programs/vault-registrar/src/utils/freeze_authority_type.rs
30:     if freeze_authority == access_control_authority_pda(&mint_key) {
34:     if freeze_authority == mint_config_pda(&mint_key) {
102:     let acl_authority = access_control_authority_pda(asset_mint);
123:         mint_config_pda(asset_mint),
147:     mint_config_pda(asset_mint),
168:     access_control_authority_pda(asset_mint),

programs/vault-registrar/src/utils/access_control.rs
28:     access_control_authority_pda(asset_mint),

programs/vault-registrar/src/instructions/register_vault_spl.rs
107: let freeze_authority_type = FreezeAuthorityType::from_mint(&ctx.accounts.asset_mint)?;
111: .require_valid(&asset_mint, freeze_authority_type)?;
115: .require_valid(&asset_mint, freeze_authority_type)?;

programs/vault-registrar/src/instructions/thaw_vault_token_account_spl.rs
81: let freeze_authority_type = FreezeAuthorityType::from_mint(&ctx.accounts.asset_mint)?;
85: .require_valid(&asset_mint, freeze_authority_type)?;
```

Recommended Mitigation: Resolve each PDA once per instruction and thread the values through the validation chain instead of re-deriving them. For example, have `FreezeAuthorityType::from_mint` also return the resolved authority and mint-config keys, or introduce a small context struct the `require_*` methods accept:

```
pub struct FreezeContext {
    pub kind: FreezeAuthorityType,
    pub acl_authority: Pubkey,
    pub mint_config: Option<Pubkey>,
}

impl FreezeContext {
    pub fn from_mint(asset_mint: &InterfaceAccount<Mint>) -> Result<Self> {
        let mint_key = asset_mint.key();
        let freeze_authority: Pubkey = asset_mint.freeze_authority.into()
            .ok_or(VaultRegistrarError::MintNotAclManaged)?;
        let acl_authority = access_control_authority_pda(&mint_key);
        if freeze_authority == acl_authority {
            return Ok(Self { kind: FreezeAuthorityType::AcProgram, acl_authority, mint_config: None });
        }
        let mint_config = mint_config_pda(&mint_key);
        if freeze_authority == mint_config {
            return Ok(Self { kind: FreezeAuthorityType::AcProgramWithSrvc37, acl_authority,
                ↪ mint_config: Some(mint_config) });
        }
        Err(VaultRegistrarError::MintNotAclManaged.into())
    }
}
```

The `require_freeze_authority_accounts`, `require_mint_config`, and `require_acl_backed_mint_config` methods then take the precomputed `acl_authority` and `mint_config` pubkeys instead of calling `find_program_address` again, collapsing the duplicated authority and mint-config derivations to one each per instruction. The distinct `access_control_state_pda` is still derived by `require_access_control_state` and is

not part of the context, so the worst-case sRFC-37 path drops from up to eight `find_program_address` calls per instruction to three.

Securitize: Fixed in commit [515847](#).

Cyfrin: Verified.